

TravelBharat

1. Project Overview

TravelBharat is a full-stack web application designed to act as a travel directory or portal. It allows users to browse through various travel destinations across India, categorized by states, cities, and specific place types. The platform also includes a comprehensive Admin Dashboard to manage all the travel data dynamically.

2. Tech Stack

The project is built using the MERN stack (MongoDB, Express, React, Node.js):

- ❖ **Frontend (Client):** * React.js (Bootstrapped with Vite, indicated by VITE_API_URL and localhost:5173).
 - State Management via React Context API (AuthContext.jsx).
 - Standard Web Technologies (JavaScript, CSS, HTML).
- ❖ **Backend (Server):** * Node.js (v18+) & Express.js (indicated by controllers, middleware, and routes structure).
 - MongoDB (Database) with Mongoose (implied by the models directory structure).
 - JWT (JSON Web Tokens) for secure authentication and authorization.

3. Core Features

- ❖ **User Exploration:**
 - Browse places by State, City, or Category.
 - Advanced filtering capabilities for places (search, state, city, category, featured status, pagination).
 - Detailed view pages for specific places (PlaceDetailPage.jsx) and states (StatePage.jsx).
- ❖ **Authentication & Authorization:**
 - User Registration and Login systems.
 - Protected routes for authenticated actions (ProtectedRoute.jsx).
 - Role-based access separating standard users from Admins.
- ❖ **Admin Dashboard:**
 - Secure Admin Login.
 - Overview statistics dashboard.
 - Full CRUD (Create, Read, Update, Delete) capabilities to manage:
 - States & Cities
 - Categories
 - Specific Travel Places

4. Application Architecture & Structure

The repository follows a clean Monorepo structure, strictly separating the frontend and backend.

- ❖ **/server (Backend):** Follows the standard MVC (Model-View-Controller) design pattern.
 - Models: Schemas for User, State, City, Place, and Category.
 - Controllers: Contains the business logic for auth, places, categories, cities, states, and admin actions.
 - Routes: Maps API endpoints to their respective controllers.
 - Middleware: Includes JWT auth.js to protect routes and a centralized errorHandler.js.
 - Database Seeding: Includes a seed.js file that populates the database with default data (6 states, 16 cities, 15 places, and 1 admin account) for easy local setup.
- ❖ **/client (Frontend):** Component-based architecture.
 - Pages: Separated into public-facing pages (Home, States, Places) and an /admin sub-directory for dashboard views.
 - Components: Reusable UI elements like Navbar, Footer, PlaceCard, StateCard, and Pagination.
 - Services: Extracted API calls (api.js, travelService.js) to keep components clean from direct HTTP request logic.

5. API Design

The backend exposes a RESTful API. Key endpoints include:

- ❖ **Public Data Routes:** /api/states, /api/cities, /api/places, /api/categories.
- ❖ **Authentication:** /api/auth/login, /api/auth/register, /api/auth/me.
- ❖ **Protected Admin Routes:** /api/admin/stats, /api/admin/places, /api/admin/states, etc., strictly used for managing the platform's data.

6. Deployment Readiness

The repository is set up with production in mind and includes clear deployment instructions:

- ❖ **Frontend Environment:** Configured to be easily deployed on Vercel via .env variables mapped to the backend.
- ❖ **Backend Environment:** Configured to be hosted on platforms like Render, hooking up to a cloud-hosted MongoDB Atlas cluster.